Programación Orientada a Objetos en JavaScript

Guía de referencia rápida y progresiva: desde objetos literales hasta herencia de clases y prototipos.

JS OOP Guide

```
const guide = new ReferenceGuide({
  target: 'JavaScript OOP',
  level: 'Advanced',
  mode: 'Editorial'
});
guide.init();
```

El comienzo de la instancia

NotebookLM

Objetos Base (Plain Objects)

Un objeto es una colección de datos agrupada en pares clave: valor. 🔑 📇

Se accede a sus valores mediante la notación de punto (obj.prop).

Para claves con espacios o caracteres especiales, se usan corchetes (obj['prop']).

```
let user = {
  name: 'John',
  age: 30,
  'likes birds': true
};

console.log(user.name);

console.log(user['likes birds']);

delete user.age;
```

Notación de punto

Notación de corchetes

NotebookLM

Propiedades Dinámicas y Atajos

Shorthand

Permite usar el nombre de una variable existente directamente como clave y valor.

Computed Properties

Usar corchetes [...] al crear un objeto evalúa una expresión para generar la clave dinámicamente.

```
function makeUser(name, age) {
  let fruit = 'apple';

  return {
    name,
    age,
    [fruit]: 5
  };
}

let user = makeUser('John', 30);
```

Equivale a name: name

NotebookLM

Iteración y Verificación

✅ El operador `in` verifica si una clave existe en el objeto, devolviendo `true` o `false` (incluso si el valor es `undefined`).

✅ El bucle `for..in` recorre todas las `clavees` existentes en un objeto para inspeccionar su contenido.

```
let user = { name: 'John', age: 30
};

if ('age' in user) {
  console.log('Age exists!');
}

for (let key in user) {
  console.log(key, '->', user[key]);
}
```

Terminal

```
> Age exists!
> name -> John
> age -> 30
```

NotebookLM

Sintaxis Básica de Clases

Una `class` es una plantilla extensible para crear múltiples objetos con la misma estructura.

El método `constructor()` se ejecuta automáticamente al instanciar con `new`.

⚠️ **Regla estricta:** No se usan comas para separar los métodos dentro de una clase.

Code

```
class User {
  constructor(name) {
    this.name = name;
  }

  sayHi() {
    console.log(`Hello, ${this.name}`);
  }
}


let user = new User('John');
user.sayHi();
```

⚠️ ¡Sin coma aquí!

NotebookLM

¿Qué es realmente una Clase?

1. A nivel interno, una clase en JavaScript es un tipo especial de función.
2. El código del constructor se convierte en el cuerpo de la función principal.
3. Todos los métodos de la clase se almacenan internamente en la propiedad User.prototype.

Code

```
class User {
  constructor(name) { this.name = name;
  }
  sayHi() { console.log('Hi'); }
}
```

Console

```
// 1. La clase es una función
> typeof User
'function'

// 2. El método está en el prototipo
> User.prototype.sayHi
[Function: sayHi]

// 3. El prototipo contiene el constructor
> User.prototype.constructor === User
true
```

NotebookLM

Campos de Clase (Class Fields)

Los campos permiten definir propiedades directamente en el cuerpo de la clase.

A diferencia de los métodos, los campos se asignan a cada objeto individual (instancia).

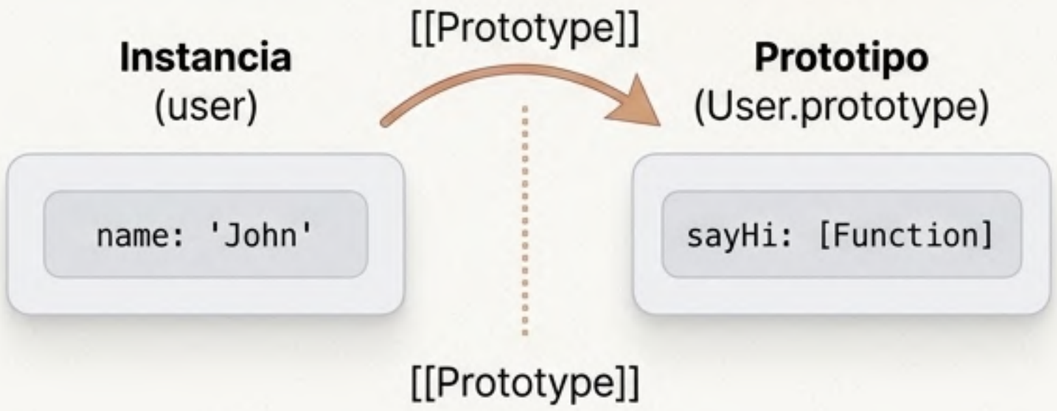
No residen en el prototipo (User.prototype), sino en el objeto mismo.

Code

```
class User {
  name = 'John';

  sayHi() {
    console.log(`Hello, ${this.name}`);
  }
}

let user = new User();
```



NotebookLM

Getters y Setters

Permiten definir propiedades de acceso que ejecutan lógica al **leer** (get) o **escribir** (set) un valor.

Técnicamente, se crean en el prototipo de la clase.

Se usan comúnmente junto con variables con guion bajo (_name) para simular propiedades internas.

Code

```
class User {
  constructor(name) {
    this.name = name;
  }

  get name() {
    return this._name;
  }

  set name(value) {
    if (value.length < 4) {
      console.log('Nombre muy corto.');
      return;
    }
    this._name = value;
  }
}
```

Esto invoca automáticamente al setter

Convención: indica propiedad interna

NotebookLM

Binding de Métodos con Campos

Si pasas un método de clase como callback (ej. setTimeout), pierde su contexto (**this** se vuelve undefined).

Solución elegante: Declarar el método como un Campo de Clase usando una arrow function.

Esto vincula permanentemente **this** a la instancia específica del objeto.

Code

```
class Button {
  constructor(value) {
    this.value = value;
  }

  click = () => {
    console.log(this.value);
  }
}

let button = new Button('he
setTimeout(button.click, 1000);
```

Vincula permanentemente 'this' a la instancia

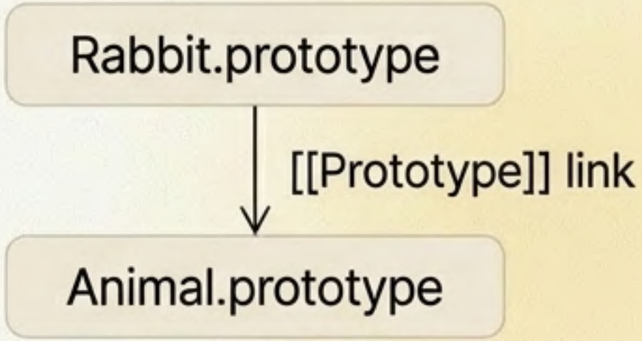
NotebookLM

Herencia de Clases (extends)

La palabra clave **extends** permite crear una clase hija basada en una clase padre.

Los objetos de la clase hija obtienen acceso a los métodos de ambas clases.

Internamente, establece el prototipo de la clase hija hacia el prototipo del padre.



Code

```
class Animal {
  run() { console.log('Corriendo...'); }
}

class Rabbit extends Animal {
  hide() { console.log('Escondido!'); }
}

let rabbit = new Rabbit();
rabbit.run();
rabbit.hide();
```

Heredado de Animal

Propio de Rabbit

NotebookLM

Sobrescritura de Métodos (super)

Una clase hija puede reemplazar un método del padre definiendo un método con el mismo nombre.

Para ampliar la funcionalidad original (en lugar de destruirla), usamos **super.metodo()** para invocar la lógica del padre.

Code

```
class Animal {
  stop() {
    console.log('Animal detenido.');
}

class Rabbit extends Animal {
  stop() {
    super.stop();
    console.log('El conejo se esconde.');
  }
}

let rabbit = new Rabbit();
rabbit.stop();
```

NotebookLM

Sobrescritura del Constructor

Si una clase hija declara su propio constructor, debe llamar a **super(...)** obligatoriamente.

La llamada a **super(...)** debe ocurrir antes de usar **this**.

Esto se debe a que el constructor padre es el responsable de inicializar el objeto base en memoria.

Code

```
class Animal {
  constructor(name) { this.name = name; }
  }
}

class Rabbit extends Animal {
  constructor(name, earLength) {
    super(name);
    this.earLength = earLength;
  }
}

let rabbit = new Rabbit('Bugs', 10);
```

⚠️ ¡Obligatorio antes de usar 'this'!

NotebookLM

Campos en Herencia: Un Comportamiento Peculiar

Si un constructor padre utiliza un campo de clase, siempre leerá su propio valor, no el sobrescrito por la hija.

Razón: El campo de la clase hija no existe en el momento en que se ejecuta el constructor del padre (**super()**).

Code

```
class Animal {
  name = 'animal';
  constructor() {
    console.log(this.name);
  }
}

class Rabbit extends Animal {
  name = 'rabbit';
}

new Rabbit();
```

Orden de Ejecución:

1. Se llama a super() (Constructor padre).
2. Se inicializa el campo 'name' de Animal.
3. Se ejecuta el **console.log** → Imprime 'animal'.
4. Se inicializa el campo 'name' de Rabbit.

NotebookLM

Comprobación de Tipos (instanceof)

El operador **instanceof** verifica si un objeto pertenece a una clase específica.

Sube por la cadena de prototipos, por lo que también considera la herencia.

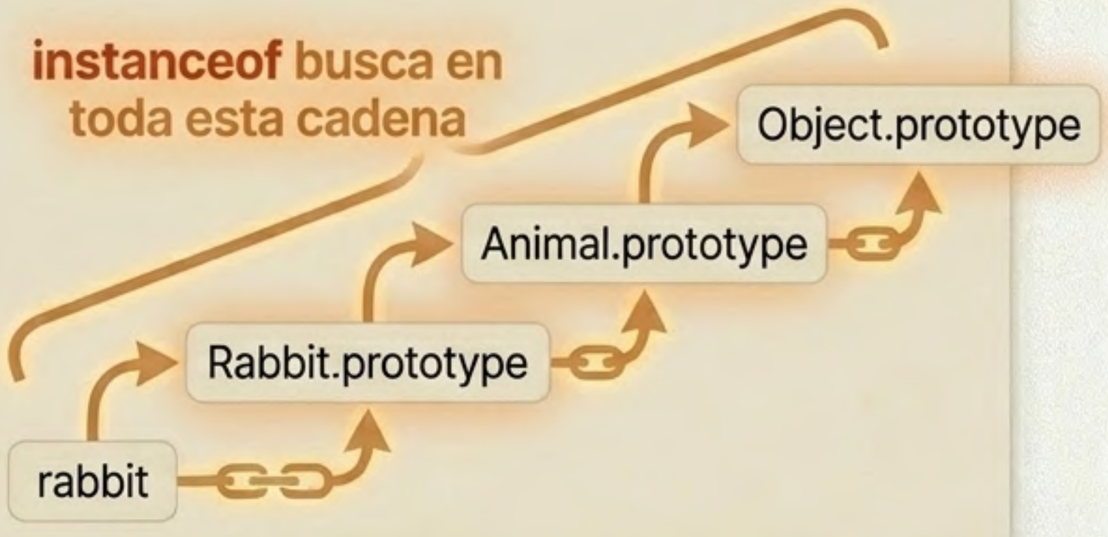
Devuelve **true** si encuentra el prototipo de la clase en cualquier parte de la cadena del objeto.

Code

```
class Animal {}
class Rabbit extends Animal {}

let rabbit = new Rabbit();

console.log(rabbit instanceof Rabbit);
console.log(rabbit instanceof Animal);
```



NotebookLM

Síntesis: Herramientas de Tipado

Selecciona la herramienta adecuada según lo que necesites inspeccionar.

typeof es ideal para primitivos.

instanceof brilla al trabajar con jerarquías de clases y herencia.

{}.toString ofrece un diagnóstico avanzado para objetos nativos.

Diagnostic Synthesis Matrix		
Herramienta	**Funciona para**	**Devuelve**
typeof	Primitivos	string
{}.toString.call()	Primitivos, Objetos Nativos, Symbol.toStringTag	string (ej. '[object Array]')
instanceof	Objetos y Clases (considera herencia)	true / false

NotebookLM